

Prueba Técnica: Desarrollador FullStack (Laravel + Angular)

Empresa: Todotek S.A.

Proyecto: Sistema de Gestión de Productos "Todostock"

1. Objetivo

Desarrollar una aplicación funcional que demuestre habilidades en el manejo del stack **Laravel (Backend)** y **Angular (Frontend)**, integrando una base de datos **PostgreSQL**. Se evaluará la arquitectura, el manejo de seguridad y la calidad del código.

2. Requerimientos Técnicos (Stack)

- **Backend:** PHP (Framework Laravel).
- **Frontend:** Framework Angular.
- **Base de Datos:** PostgreSQL.
- **Autenticación:** Implementar generación de tokens (Laravel Sanctum).
- **Seguridad :** Protección de rutas y auditoría de operaciones críticas

3. Funcionalidades del Modelo de Datos

La prueba consiste en desarrollar un sistema CRUD avanzado enfocado en **Productos y Clientes**, con los siguientes requerimientos:

A. Gestión de Productos (CRUD Completo)

- **Creación:** Formulario mediante ventana **Modal**.
- **Edición:** Formulario dedicado o modal para actualización de datos.
- **Listado:** Tabla con soporte para **filtros** (por nombre, categoría, etc.) y **paginación** del lado del servidor.
- **Detalle y Eliminación:** Capacidad de visualizar la ficha completa del producto y eliminar registros con confirmación.
- **Control de Stock:** Lógica para manejar existencias (entradas/salidas).
- **Impuestos:** Implementación de cálculo de impuestos (IVA u otros cargos configurables).

B. Gestión de Imágenes

- **Carga Múltiple:** Permitir subir varias imágenes por producto.

- **Almacenamiento:** Uso del sistema de archivos (Storage/Disco).
- **Gestión:** Capacidad de visualizar las imágenes subidas y eliminar imágenes específicas de un producto.

C. Facturación y Clientes

- **CRUD de Clientes:** Gestión básica de los datos de clientes.
- **Módulo de Factura:** Generación de un registro de venta que relacione cliente, productos seleccionados y el cálculo final (Subtotal, Impuestos, Total).

4. Interfaz de Usuario y Experiencia (UX/UI)

- **Notificaciones:** Uso obligatorio de alertas dinámicas para confirmar acciones o reportar errores (ej. **SweetAlert2** o **Toast notifications**).
- **Diseño:** Debe ser limpio, responsivo y funcional.
- **Menú:** Menú lateral izquierdo con las opciones del sistema.

5. Criterios de Evaluación

1. **Arquitectura de Software:** Uso correcto de controladores, modelos y servicios.
2. **Seguridad:** Manejo de rutas protegidas mediante tokens.
3. **Manejo de Datos:** Consultas eficientes a PostgreSQL y validaciones de datos (Requests).
4. **Frontend:** Componentización en Angular y manejo de estados.

6. Repositorio de Código (GitHub / GitLab / Bitbucket)

Se recomienda un único repositorio con dos carpetas principales o dos repositorios vinculados:

- **/backend:** Proyecto en Laravel.
- **/frontend:** Proyecto en Angular.

7. Documentación (Archivo README.md)

Un buen README es la carta de presentación. Debe incluir:

- **Guía de Instalación:** Pasos exactos para levantar el proyecto (ej. `composer install`, `npm install`, `php artisan migrate`).
- **Variables de Entorno:** Ejemplo de archivo `.env` (sin las claves reales) indicando qué parámetros de base de datos y JWT se necesitan.

- **Documentación de la API:** Lista de los endpoints principales (pueden usar herramientas como **Postman Collection** o **Swagger** integrado en Laravel).
- **Supuestos Técnicos:** Si el candidato tomó una decisión específica (ej. "Usé el IVA del 15% como constante configurada en el backend"), debe explicarlo aquí.

3. Base de Datos y Scripts SQL

Aunque Laravel usa *Migrations* incluir

- **Migrations & Seeders:** Archivos de Laravel que creen las tablas de Productos, Clientes, Facturas e Imágenes, y que incluyan datos de prueba (Seeders) para ver la paginación de inmediato.
- **Script SQL (Opcional pero recomendado):** Un archivo .sql con el dump de la base de datos PostgreSQL por si el evaluador prefiere importarla directamente.

4. Entregables de Funcionalidad Específica

Adjuntar:

- **Carpeta de Almacenamiento:** Asegurarse de que la lógica de Storage de Laravel esté configurada para que las imágenes subidas sean visibles.
- **Colección de Postman:** Un archivo JSON con las peticiones listas para probar el login (token), el CRUD y la facturación.

5. Demostración de Notificaciones (UX)

En la documentación o mediante capturas de pantalla, el candidato debe demostrar que implementó:

- **SweetAlert2 / Toasts:** Para confirmar cuando un producto se elimina o cuando el stock llega a cero.

Check-list de Entrega:

1. ☐ **Repositorio Público o Acceso compartido:** URL del código.
2. ☐ **Backend:** Código en Laravel + Sanctum/JWT.
3. ☐ **Frontend:** Código en Angular + componentes Modal.
4. ☐ **Base de Datos:** Migraciones de PostgreSQL y Seeders.
5. ☐ **README:** Instrucciones de "Quick Start".
6. ☐ **Postman:** Colección de endpoints para pruebas rápidas.